

NovaTareas Pro

Observabilidad, rendimiento y escalabilidad

Campo	Detalle
Evaluación	Semana 5 — Módulo 4: Desarrollo de Aplicaciones con IA
Tema	Instrumentación, línea base de rendimiento y plan de escalabilidad
Equipo	Equipo 3
Integrantes	Saúl Oswaldo López Hernández — SMIS108421 Moises Antonio Martínez — SMIS071221 Enson Onan Carranza Rodríguez — SMIS013020
Repositorio	https://github.com/Saul1hdz/NovaTareas
Fecha	14 de agosto de 2026

Este documento presenta la instrumentación incorporada a NovaTareas Pro, la línea base de rendimiento medida sobre su flujo crítico, el cuello de botella identificado con evidencia y el plan de escalabilidad que se deriva de él. Todas las cifras se obtuvieron ejecutando el proyecto real de este repositorio y se pueden reproducir con los comandos incluidos.

1. Servicio y flujo crítico

NovaTareas Pro es un gestor de tareas con asistente de productividad. Corre como aplicación Astro en modo servidor (adaptador Node) contra PostgreSQL 16, y expone además una API pública `/api/v1` con el motor de recomendaciones basado en z.ai (GLM), con respaldo local en Ollama y reglas heurísticas.

Flujo crítico seleccionado: GET `/api/tasks`

Es la petición que más importa al usuario y la que más se ejecuta: el dashboard la lanza al cargar, al filtrar, al buscar y después de crear, completar, archivar o borrar una tarea. Si esta ruta se degrada, la aplicación entera se siente lenta aunque el resto funcione.

La consulta no es trivial: agrega las subtareas con `STRING_AGG` y trae la última recomendación de IA de cada tarea con un `LEFT JOIN LATERAL`, filtrando por usuario y estado de archivado.

Como **control** se mide también `GET /api/v1/health`, que no consulta la base ni valida sesión. Sin esa referencia, cualquier latencia se atribuiría por defecto a la consulta, que es justamente el error que este trabajo evita cometer.

2. Preguntas de observabilidad y campos registrados

La instrumentación se diseñó a partir de las preguntas que hay que poder responder cuando algo va mal, no al revés.

#	Pregunta que debe poder responderse	Campo del evento
1	¿De qué solicitud concreta habla quien reporta el fallo?	<code>request_id</code>
2	¿Qué funcionalidad se ejecutó?	<code>route</code> , <code>method</code>
3	¿Terminó bien, lo rechazamos o se rompió el servidor?	<code>status</code> , <code>outcome</code>
4	¿Cuánto tardó en total?	<code>duration_ms</code>
5	¿Qué parte fue base de datos y cuántas consultas costó?	<code>db_duration_ms</code> , <code>db_queries</code>
6	¿Qué componente inteligente respondió, con qué modelo y prompt?	<code>ai_component</code> , <code>ai_source</code> , <code>ai_model</code> , <code>ai_prompt_version</code>
7	¿Hubo que degradar a un respaldo porque el proveedor falló?	<code>ai_fallback</code> , <code>ai_duration_ms</code>
8	Cuando falló, ¿de qué tipo fue el fallo?	<code>error_type</code>
9	¿Qué versión del código lo produjo?	<code>app_version</code>

3. Código de la instrumentación

3.1 Contrato del evento — `src/lib/observability.js`

Los campos publicables son una **lista blanca**. Lo que no está aquí no sale, venga de donde venga: la privacidad es una propiedad estructural, no una disciplina que haya que recordar en cada llamada.

```
const EVENT_FIELDS = [
  'ts', 'level', 'event', 'request_id', 'method', 'route', 'status',
  'outcome', 'duration_ms', 'app_version',
  'ai_component', 'ai_source', 'ai_model', 'ai_prompt_version',
  'ai_duration_ms', 'ai_fallback',
  'error_type', 'db_queries', 'db_duration_ms',
];
```

```
function buildEvent(fields) {
  const event = {};
  for (const key of EVENT_FIELDS) {
    if (fields[key] !== undefined && fields[key] !== null) event[key] = fields[key];
  }
  return event;
}
```

3.2 Medición de toda solicitud — src/middleware.js

Vive en el middleware y no en cada endpoint: así ninguna ruta puede olvidarse de registrar su evento, y el `request_id` queda disponible para el código anidado mediante `AsyncLocalStorage`, sin pasarlo por parámetros.

```
export const onRequest = defineMiddleware(async (context, next) => {
  const pathname = new URL(context.request.url).pathname;
  if (!shouldLogPath(pathname)) return next();

  const requestId = resolveRequestId(context.request.headers.get('x-request-id'));
  const startedAt = performance.now();

  return runWithRequestContext({ request_id: requestId }, async () => {
    let response, thrown = null;
    try { response = await next(); } catch (error) { thrown = error; }

    const status = thrown ? 500 : response.status;
    if (thrown) annotate({ error_type: errorTypeOf(thrown) });

    logRequestEvent({
      ...currentContext(),
      level: status >= 500 ? 'error' : 'info',
      request_id: requestId,
      method: context.request.method,
      route: normalizeRoute(pathname),
      status,
      outcome: outcomeFor(status),
      duration_ms: Math.round(performance.now() - startedAt),
    });

    if (thrown) throw thrown;
    response.headers.set('x-request-id', requestId);
    return response;
  });
});
```

3.3 Coste real de PostgreSQL — src/db/client.js

Único punto por el que pasan todas las consultas. Se mide el tiempo, nunca el SQL ni los parámetros: esos llevan datos del usuario.

```
async query(text, values = []) {
  const startedAt = performance.now();
  try {
    return await (this.client || this.pool).query(text, values);
  } finally {
    recordDbQuery(performance.now() - startedAt);
  }
}
```

3.4 Componente inteligente — src/lib/aiEngine.js

```
const report = (source, model, fallback) => annotate({
  ai_component: 'recommendation-engine',
  ai_source: source, // zai | ollama | rules
  ai_model: model,
  ai_prompt_version: PROMPT_VERSION, // 'recommend-v1'
  ai_duration_ms: Math.round(performance.now() - startedAt),
  ai_fallback: fallback,
});
```

4. Evidencias de ejecución

4.1 Evento exitoso

```
$ curl -s -o /dev/null -D - "http://127.0.0.1:4321/api/tasks?archived=0" \
-H "Cookie: $COOKIE"
HTTP/1.1 200 OK
x-request-id: a361aae4-3edf-4469-94d3-bcb645de4800

{"ts":"2026-08-14T02:52:35.328Z","level":"info","event":"http_request",
 "request_id":"a361aae4-3edf-4469-94d3-bcb645de4800","method":"GET",
 "route":"/api/tasks","status":200,"outcome":"success","duration_ms":24,
 "app_version":"dev","db_queries":2,"db_duration_ms":18.93}
```

Interpretación. El identificador devuelto al cliente en la cabecera `x-request-id` es exactamente el mismo que aparece en el log: quien reporta un problema puede citarlo y el evento se localiza sin buscar por hora aproximada. La solicitud tardó 24 ms, de los cuales 18,93 ms fueron PostgreSQL repartidos en 2 consultas. Es la primera petición tras reiniciar el contenedor, así que incluye abrir el pool de conexiones; en régimen estable esa cifra baja a ~3 ms, como muestra la sección 5.

4.2 Error controlado

```
$ curl -s -D - "http://127.0.0.1:4321/api/tasks?priority=inventada" \
-H "Cookie: $COOKIE"
HTTP/1.1 400 Bad Request
x-request-id: 6e4ce0b3-4331-40b5-a4a8-e829b0835b8b
{"error":"Prioridad invalida"}

{"ts":"2026-08-14T02:52:35.477Z","level":"info","event":"http_request",
 "request_id":"6e4ce0b3-4331-40b5-a4a8-e829b0835b8b","method":"GET",
 "route":"/api/tasks","status":400,"outcome":"client_error","duration_ms":4,
 "app_version":"dev","error_type":"prioridad_invalida","db_queries":1,
 "db_duration_ms":1.32}
```

Interpretación. El rechazo es deliberado, no una excepción: `outcome` lo clasifica como `client_error` y `error_type` dice **por qué** se rechazó, de modo que los 400 se pueden contar por causa en lugar de formar un montón indistinguible. El valor inválido enviado por el cliente no aparece en el log. Un detalle que solo se ve gracias a la instrumentación: la petición rechazada gasta igualmente una consulta a la base, la de validación de sesión, que ocurre antes que la del filtro.

5. Datos excluidos por privacidad y seguridad

Dato excluido	Motivo
Cuerpos de solicitud y respuesta	Llevan títulos, descripciones y comentarios del usuario
Cabeceras Authorization y Cookie	Contienen el token de sesión y la clave de la API
Contraseñas y respuestas de seguridad	Nunca salen de su hash
Cadena de conexión y SQL con parámetros	Revelaría credenciales y datos de usuario
Correo, nombre y teléfono	Datos personales innecesarios para diagnosticar
Query string	Un término de búsqueda es contenido del usuario
Tokens de invitación en la ruta	<code>normalizeRoute</code> los sustituye por <code>:token</code>
Mensaje de la excepción	Puede arrastrar valores; solo se publica la clase

Esto está cubierto por pruebas automatizadas en `tests/observability.test.js`, que verifican que campos como `authorization`, `password`, `email`, `cookie` o `body` se descartan aunque alguien los pase explícitamente al logger, y que un `x-request-id` entrante con saltos de línea no puede inyectar eventos falsos.

6. Escenario de la medición

Parámetro	Valor
Ambiente	Docker Desktop (WSL 2) sobre Windows 11 Pro; cliente en el host
Servicios	web (Node 22) + db (PostgreSQL 16), red interna de Compose
Endpoint	GET /api/tasks?archived=0 con sesión iniciada
Datos	Cuenta ficticia ana.demo@example.test, 12 tareas en la base
Autenticación	Cookie de sesión obtenida vía POST /api/login
Solicitudes	30 secuenciales por tanda; 40 en las tandas con concurrencia
Calentamiento	3 solicitudes descartadas antes de medir
Herramienta	scripts/measure-endpoint.mjs (incluido en el repositorio)
Versión del código	Rama semana5-observabilidad; app_version dev / build-semana5
Componente IA	recommend-v1, modelo glm-4.5-flash (no interviene en este endpoint)

```
MEASURE_USER_EMAIL=ana.demo@example.test MEASURE_USER_PASSWORD=... \  
node scripts/measure-endpoint.mjs --scenario=tasks --requests=30
```

7. Línea base de rendimiento

7.1 Servidor de desarrollo (astro dev)

Escenario	p50 (ms)	p95 (ms)	Máx (ms)	Error	req/s
tasks c=1	20,67	24,56	24,68	0 %	47,9
tasks c=4	45,45	142,69	159,15	0 %	69,7
tasks c=8	79,53	135,00	148,11	0 %	90,7
tasks c=16	218,09	298,35	379,87	0 %	65,7
health c=1 (control)	14,02	20,87	—	0 %	65,9
health c=8 (control)	52,13	79,25	—	0 %	137,5

Tanda secuencial: mínimo 17,45 ms y promedio 20,82 ms. Tasa de error 0 % en todas las tandas: 30/30 y 40/40 respuestas HTTP 200.

7.2 Servidor compilado (node dist/server/entry.mjs)

Escenario	p50 (ms)	p95 (ms)	Máx (ms)	Error	req/s
tasks c=1	16,19	23,74	49,39	0 %	56,4
tasks c=4	31,36	87,60	109,83	0 %	104,4
tasks c=8	73,37	208,76	227,88	0 %	93,0
tasks c=16	172,05	291,81	546,09	0 %	71,8

7.3 Lo que mide el cliente frente a lo que tarda el servidor

La instrumentación permite comparar ambos lados sobre las mismas 30 solicitudes secuenciales del servidor compilado.

Medida	p50	p95	Máx
Cliente (measure-endpoint.mjs)	14,94 ms	17,68 ms	17,78 ms
Servidor (duration_ms del log)	4 ms	6 ms	7 ms
PostgreSQL (db_duration_ms)	2,97 ms	4,20 ms	—

Interpretación. Consultas por solicitud: 2, constante. El servidor cree tardar 4 ms y el cliente observa 15: la diferencia no está en el código.

8. Diagnóstico: cuello de botella

8.1 Dónde se va el tiempo

Tramo	Tiempo	Porcentaje
Fuera de la aplicación (red de Docker Desktop en Windows + cliente)	~10,9 ms	73 %
Aplicación sin base de datos	~1,0 ms	7 %
PostgreSQL (2 consultas)	~3,0 ms	20 %

El cuello de botella de la latencia percibida no está en el código. Casi tres cuartas partes se consumen antes de que la solicitud llegue al proceso Node: el reenvío de puertos de Docker Desktop entre Windows y la VM de WSL 2. Se confirmó midiendo desde dentro del contenedor, donde el mismo endpoint de control baja de ~17,8 ms a ~13,4 ms de p50.

El endpoint crítico añade solo ~6 ms sobre el control, que no toca la base ni valida sesión. **La consulta con STRING_AGG y LEFT JOIN LATERAL no es el problema** con el volumen actual de datos, que era la hipótesis intuitiva y resultó falsa.

8.2 Segundo hallazgo: la mitad de las consultas son de sesión

db_queries vale 2 en toda solicitud autenticada, y una de ellas no es de negocio: getUser() valida la sesión con un SELECT session_version FROM users en cada petición. Se paga incluso en peticiones que se van a rechazar, como muestra el evento del error controlado con db_queries: 1. Es correcto por seguridad —permite invalidar sesiones al instante al cambiar la contraseña— pero duplica el coste de base de datos por solicitud.

8.3 Comportamiento bajo carga

El throughput se estanca entre ~65 y ~105 req/s mientras la latencia crece de forma aproximadamente proporcional a la concurrencia: de c=1 a c=16 el p50 pasa de 16 a 172 ms, unas 10,6 veces con 16 veces más carga. Eso es saturación: más allá de 4 peticiones simultáneas, el tiempo añadido es cola de espera, no trabajo. El techo es prácticamente el mismo para el control que para el flujo crítico, y para el servidor compilado que para el de desarrollo, lo que descarta la consulta y la compilación como causa. **El límite es la capacidad de un único proceso Node en un contenedor.**

8.4 Riesgo detectado en la propia medición

Las mediciones desde el host tienen varianza alta: dos tandas idénticas del control dieron 14,02 ms y 19,40 ms de p50. Es propio de Docker Desktop en Windows. La conclusión operativa es que para decidir sobre rendimiento hay que usar `duration_ms` del log y no el cronómetro del cliente: la instrumentación no es solo diagnóstico, es la métrica fiable.

9. Mejora aplicada y comparación antes/después

Mejora aplicada: servir el proyecto compilado en lugar del servidor de desarrollo. El contenedor ejecutaba `npm run dev` —Vite transformando módulos en cada solicitud— también cuando se enseñaba la demo.

Métrica (tasks c=1)	astro dev	Compilado	Cambio
p50	20,67 ms	16,19 ms	-21,7 %
p95	24,56 ms	23,74 ms	-3,3 %
Throughput	47,9 req/s	56,4 req/s	+17,7 %

Interpretación. La mejora se confirmó en dos ejecuciones independientes (-21 % y -22 %), así que no es ruido de medición. En cambio no mejora el techo bajo carga, coherente con el diagnóstico: la saturación es del proceso, no de la compilación.

```
docker compose -f compose.dev.yml exec web npm run build
docker compose -f compose.dev.yml stop web
docker compose -f compose.dev.yml run -d --rm --service-ports \
  --name novatareas-build -e APP_VERSION=build-semana5 \
  web node ./dist/server/entry.mjs
```

Mejora propuesta, no aplicada: cachear la validación de sesión unos segundos en memoria para eliminar la mitad de las consultas por solicitud. No se aplica porque tiene un coste de seguridad real —retrasaría la invalidación inmediata de sesiones al cambiar la contraseña— y porque la evidencia dice que ahorraría ~1,5 ms sobre 15: no compensa hoy. Se reconsiderará cuando la base deje de estar en la misma máquina y cada consulta cueste latencia de red.

10. Plan de escalabilidad

Crecimiento considerado. Demo cerrada de uso académico: decenas de usuarios ficticios con picos durante una presentación. El escenario realista es 20–50 personas abriendo el dashboard a la vez, no tráfico sostenido.

Restricción observada. Un proceso Node satura entre 65 y 105 req/s y la latencia crece linealmente a partir de 4 peticiones concurrentes. Con 50 usuarios simultáneos el p50 estimado supera los 400 ms.

Orden de las mejoras, por rendimiento sobre esfuerzo

#	Mejora	Por qué en esta posición
1	Servir siempre el build	Ya aplicado y medido: -21 % de p50, sin coste ni infraestructura
2	No usar Docker Desktop de Windows como entorno de servicio	El 73 % de la latencia observada es su reenvío de puertos; desaparece en el despliegue Linux
3	Escalar horizontalmente el contenedor web	El límite medido es de proceso. El estado compartido ya vive en PostgreSQL, así que admite varias instancias sin trabajo previo
4	Cachear la recomendación de IA, no la lista de tareas	Un GET /api/tasks cuesta 3 ms de base; una llamada a z.ai cuesta segundos y dinero

Cuándo usar cada recurso, y con qué indicador

Recurso	Cuándo	Indicador que lo dispara
Más instancias de web	Antes que nada, si sube la concurrencia	duration_ms p95 > 300 ms con CPU alta
Caché de recomendaciones	Al repetirse peticiones de IA equivalentes	ai_duration_ms domina duration_ms
Cola o workers	Solo si la IA pasa a ser síncrona y masiva	ai_duration_ms p95 > 5 s sostenido
Réplica de lectura	Cuando la base deje de ser el 20 % del tiempo	db_duration_ms > 50 % de duration_ms

Impacto en memoria, datos, privacidad y costo

Dimensión	Impacto
Memoria	Cada instancia añade su proceso Node y su pool. Con PG_POOL_MAX=10, cuatro instancias son 40 conexiones: hay que subir max_connections o bajar el pool por instancia
Datos	No cambia el modelo. No hay estado en memoria que replicar
Privacidad	Una caché de recomendaciones guardaría texto derivado de tareas del usuario: debe vivir en la base con el mismo control de propiedad que task_recommendations, nunca en un servicio externo
Costo	Escalar horizontalmente en el mismo servidor es coste cero hasta agotar CPU. El gasto real es z.ai por llamada, y ahí la caché ahorra dinero además de tiempo

Indicador para decidir cuándo escalar. duration_ms p95 de /api/tasks por encima de **300 ms** durante 5 minutos, medido en el log del servidor y no en el cliente. Es accionable porque distingue la causa: si db_duration_ms acompaña, el problema es la base; si no, es el proceso, y toca añadir instancias.

11. Limitaciones pendientes

#	Limitación
1	La línea base no cubre el camino con z.ai: medir 30 recomendaciones reales consume saldo del proveedor y AGENTS.md exige autorización expresa. La instrumentación de IA está implementada y probada, pero su línea base numérica queda pendiente de esa autorización
2	POST /api/v1/recommend no se midió porque requiere AI_API_KEY, no definida en el entorno local; sin ella el endpoint responde 503. El escenario ya está implementado en el script
3	El ambiente medido es de desarrollo sobre Windows. Las cifras absolutas no representan al despliegue Linux; sirven para comparar entre sí, que es su uso aquí
4	Varianza alta entre tandas desde el host. Las conclusiones se apoyan en diferencias reproducidas dos veces o en métricas de servidor, no en una tanda aislada
5	El middleware no se ejercita en las pruebas automatizadas, que invocan los handlers directamente. Se prueban en su lugar las funciones de observability.js, incluida la garantía de redacción

12. Repositorio actualizado

<https://github.com/Saul1hdz/NovaTareas>

Qué pide la evaluación	Dónde está en el repositorio
Código fuente actualizado	Todo el árbol; rama semana5-observabilidad
Middleware / función de instrumentación	src/middleware.js, src/lib/observability.js, src/db/client.js, src/lib/aiEngine.js
Script para medir rendimiento	scripts/measure-endpoint.mjs (npm run measure)
README con instrucciones de medición	README.md, sección 18
.env.example sin claves reales	.env.example (solo marcadores de posición)
Resultados y evidencias	docs/OBSERVABILIDAD_SEMANA_5.md y docs/mediciones/*.json
Pruebas	tests/observability.test.js, tests/measureEndpoint.test.js (npm test)
Pipeline y despliegue de semanas previas	.github/workflows/, compose.prod.yml, docs/DESPLIEGUE.md

Cómo reproducir todo

```
# 1. Entorno (sin el bot: su instancia de produccion usa el mismo token)
docker compose -f compose.dev.yml up -d db web

# 2. Pruebas, incluidas las de observabilidad y estadística
npm test

# 3. Línea base del flujo crítico
MEASURE_USER_EMAIL=ana.demo@example.test MEASURE_USER_PASSWORD=... \
  node scripts/measure-endpoint.mjs --scenario=tasks --requests=30

# 4. Comportamiento bajo carga
node scripts/measure-endpoint.mjs --scenario=tasks --requests=40 --concurrency=8

# 5. Control sin base de datos ni sesión
node scripts/measure-endpoint.mjs --scenario=health --requests=40

# 6. Ver los eventos
docker compose -f compose.dev.yml logs web | grep http_request
```

Los resultados en bruto de cada ejecución quedan en docs/mediciones/. El análisis completo está en docs/OBSERVABILIDAD_SEMANA_5.md.